Universidad de San Carlos de Guatemala
Escuela de Ciencias Físicas y Matemáticas

Redes Neuronales de Aprendizaje Automático para la Inferencia de Modelos Físicos

Informe Final de Año de Prácticas elaborado por
Cesar Alejandro Barillas Segura
presentado ante el Departamento de Física

Supervisado por
Dr. Giovanni Ramírez García (ECFM-USAC)

Guatemala, 6 de noviembre de 2026

Resumen

En este trabajo se estudió si una red neuronal entrenada con datos experimentales de caída libre permite recuperar el modelo físico que los genera, en particular el valor de la aceleración de la gravedad. Para esto se grabaron 609 clips de video de un cuerpo en caída libre y se digitalizaron sus trayectorias. Si a estos datos se les aplica el método clásico, que consiste en ajustar una parábola a cada trayectoria, se obtiene una gravedad aproximadamente 3.9 % menor que la esperada. Esa diferencia se puede atribuir al montaje experimental y limita la exactitud que puede alcanzar cualquier modelo que use estos datos.

Se entrenaron redes neuronales de distintos tamaños, con tres formas diferentes de presentar los datos de entrada. La red más grande, cuando cada trayectoria se describe desde su propio punto de partida y con su velocidad inicial, predice bien la posición en clips que no vio durante el entrenamiento, y la gravedad que se obtiene de ella difiere apenas un 0.2 % de la que da el método clásico con los mismos datos. También se encontró que la forma en que se presentan los datos pesa más que el tamaño de la red. Si las entradas no permiten distinguir una trayectoria de otra, usar una red más grande no mejora la estimación de la gravedad; en cambio, cuando sí lo permiten, la red más grande es la que da mejores resultados. Con esto se confirma el supuesto planteado en el anteproyecto.

Además se analizó cuántas épocas conviene entrenar y qué aprendió la red por dentro. Después de unas 200 épocas el tiempo de entrenamiento sigue aumentando, pero la estimación de la gravedad ya no mejora. Por otro lado, dentro de la red no aparece la ecuación de movimiento de forma explícita, sino que la física está en el comportamiento de todas las neuronas juntas y no en una en particular.

En la sección 1 se describe la institución y en la sección 2 el grupo de trabajo en el que se realizaron las prácticas. La introducción (sección 3) plantea la pregunta del trabajo y reúne la teoría que se usa después: el movimiento rectilíneo uniformemente variado, cómo está formada una red neuronal y cómo aprende, las herramientas matemáticas del entrenamiento y la biblioteca PyTorch. Siguen los objetivos y la justificación. En la metodología (sección 6) se explica el montaje experimental y la toma de datos, el preprocesamiento, las formulaciones y arquitecturas que se compararon y el protocolo con que se evaluaron. La sección 7 presenta los resultados: la gravedad que recupera cada red, su error de predicción, el efecto del número de épocas y lo que se encontró al revisar la red por dentro. Al final se discuten las limitaciones del estudio, se enumeran las conclusiones y se proponen las líneas de trabajo futuro.

Índice

Resumen **0**

1. Descripción general de la Institución **3**

2. Descripción del grupo de trabajo **3**

3. Introducción **4**
3.1. Movimiento rectilíneo uniformemente variado 4
3.2. Red Neuronal 5
3.2.1. Neurona 5
3.2.2. Función de activación 5
3.2.3. Capas 6
3.2.4. Función de coste 6
3.3. Herramientas Matemáticas 7
3.3.1. Suma ponderada 7
3.3.2. Mínimos cuadrados y ajuste polinómico 7
3.3.3. Coeficiente de determinación 8
3.3.4. Derivadas parciales 8
3.3.5. Regla de la cadena 8
3.3.6. Normalización de los datos 9
3.4. Aprendizaje automático 9
3.4.1. Backpropagation 9
3.4.2. Sobreajuste y validación 9
3.4.3. Identificabilidad del modelo 9
3.5. PyTorch 10

4. Objetivos **10**
4.1. General 10
4.2. Específicos 10

5. Justificación **11**

6. Metodología **11**
6.1. Montaje experimental y toma de datos 11
6.2. Preprocesamiento 12
6.3. Formulaciones evaluadas 12
6.4. Arquitecturas evaluadas 13
6.5. Líneas base 13
6.6. Protocolo de evaluación 13
6.7. Recursos computacionales 14

7. Resultados **15**
7.1. Caracterización del conjunto de datos 15
7.2. Recuperación del parámetro físico 16
7.3. Error de predicción 18
7.4. Representación de las entradas y capacidad de la red 19
7.5. Coste y beneficio del número de épocas 19

7.6. Inspección de la red entrenada 22
7.6.1. La red simple, escrita a mano 23
7.6.2. Las derivadas de la función aprendida 23
7.6.3. La ecuación que la red aprendió, contra los datos 24
7.6.4. Cómo está repartida la representación 25
7.7. Síntesis 25

8. Limitaciones del estudio **26**

9. Conclusiones **27**

10. Trabajo futuro **28**

11. Bibliografía **29**

1. Descripción general de la Institución

La Universidad de San Carlos de Guatemala (USAC) es la institución de educación superior estatal, autónoma, con cultura democrática, con enfoque multi e intercultural, vinculada y comprometida con el desarrollo científico, social, humanista y ambiental, con una gestión actualizada, dinámica, efectiva y con recursos óptimamente utilizados para alcanzar sus fines y objetivos. Es la universidad más antigua de Guatemala, habiendo sido fundada el 31 de enero de 1676. El campus central se encuentra ubicado en el Departamento de Guatemala, Ciudad Universitaria 11 Avenida, Zona 12. Su lema es *Id y enseñad a todos*.

La Escuela de Ciencias Físicas y Matemáticas (ECFM) es una escuela no facultativa formada en el año 2009 en el evento CONVERCIENCIA. Actualmente, la escuela cuenta con la carrera de Licenciatura en Física Aplicada y Licenciatura en Matemática Aplicada. La escuela cuenta también con un Instituto dedicado a la investigación de las ciencias físicas y matemáticas (IFIM) el cual ha presentado diversas investigaciones y publicado una variedad de artículos relacionados con la divulgación del conocimiento de las ciencias.

El IFIM es la unidad de la ECFM que promueve y realiza estudios avanzados en áreas científicas fundamentales y aplicadas de las ciencias físicas y matemáticas. Las áreas y líneas de investigación del Instituto son

1. Física de altas energías, cosmología y astrofísica: Física de partículas, Astrofísica de rayos gamma.

2. Ciencias no lineales y sistemas complejos: Materia condensada, Modelos numéricos del clima, Modelos matemáticos en ciencias de la salud.

3. Geofísica: Geodesia aplicada a la tectónica.

4. Física experimental: Detección de partículas e instrumentación electrónica, Redes de sensores.

5. Física de radiaciones: Control de calidad en radioterapia.

Además, el Instituto realiza actividades de difusión y divulgación, Seminarios científicos, Conferencias de divulgación científica y también Talleres y cursos cortos.

2. Descripción del grupo de trabajo

La Física de Materia Condensada es un área muy amplia e interesante. Mediante el estudio de las propiedades fundamentales de la materia se pueden observar fenómenos interesantes que permiten el desarrollo de nuevas tecnologías. En años recientes la relación entre estos fenómenos a nivel subatómico y las propiedades generales de los materiales ha permitido el desarrollo de materiales con nuevas propiedades o materiales de diseño.

Por otro lado, uno de los desarrollos más excitantes de la Física en los últimos años es la convergencia entre la Física de la Materia Condensada y las áreas de Información Cuántica y Computación Cuántica, lo que ha permitido un avance acelerado en ambos campos llevando de la mano a la teoría y al experimento. El principal motivo de esta convergencia es el estudio de los sistemas cuánticos de muchos cuerpos donde es indispensable entender el entrelazamiento cuántico que resulta ser el recurso principal para la teoría de información cuántica.

El Grupo de investigación se enfoca en las áreas de la física de la materia condensada, incluyendo varios grupos de estudio aplicados a mecánica estadística, mecánica cuántica, óptica, nanociencia, información cuántica, computación cuántica, etc.

3. Introducción

En este proyecto se aprendió a programar, entrenar y estudiar las características internas de una red neuronal dedicada a la identificación de modelos físicos. El caso de estudio es la caída libre de un cuerpo, descrita por el movimiento rectilíneo uniformemente variado (MRUV) que se detalla en la sección 3.1. La parte técnica y descriptiva de las redes neuronales, su arquitectura, sus funciones de activación y sus funciones de coste, se trata en la sección 3.2. Los conocimientos matemáticos necesarios para construir y entrenar la red (suma ponderada, mínimos cuadrados, derivadas parciales y regla de la cadena) se resumen en la sección 3.3, y el mecanismo de aprendizaje en la sección 3.4. La implementación se realizó en PyTorch (sección 3.5).

La pregunta precursora del trabajo no es si una red puede ajustar una curva, cosa que se sabe desde hace décadas. Lo que interesa aquí es si de una red ya entrenada puede extraerse el modelo físico subyacente. Planteado de forma concreta: si se entrena una red con trayectorias reales de caída libre y después se ajusta una parábola a sus predicciones, ¿aparece el valor correcto de g? Ese fue el criterio de éxito que se adoptó, y para responderlo hacen falta un montaje experimental que produzca datos suficientes (sección 6.1), una formulación del problema de aprendizaje en la que la pregunta tenga respuesta (sección 6.3) y un protocolo de evaluación que sea válido (sección 6.6).

La respuesta que se obtuvo es afirmativa: la red recupera la gravedad con una discrepancia del 0.2 % respecto de lo que el método clásico obtiene de esos mismos datos (sección 7). El informe también documenta cuánto aporta cada ingrediente por separado, la representación de las entradas por un lado y la capacidad expresiva de la red por otro, que al final resultó ser la parte más instructiva del estudio.

3.1. Movimiento rectilíneo uniformemente variado

Un cuerpo que se mueve en línea recta con aceleración constante a describe la posición

$$y(t) = y_0 + v_0\, t + \frac{1}{2} a\, t^2, \tag{1}$$

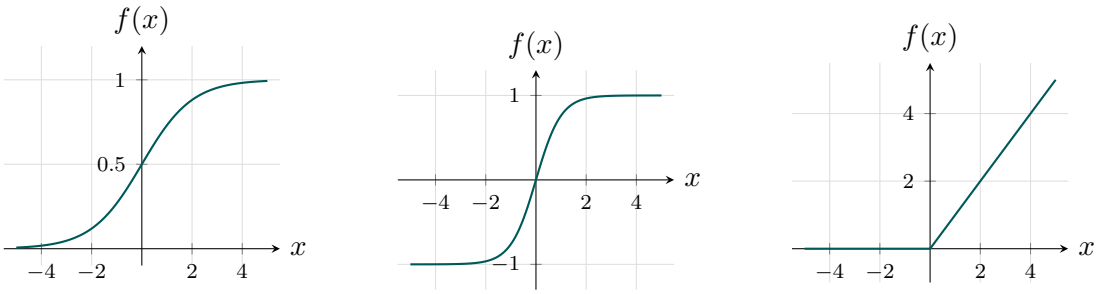
donde y_0 y v_0 son la posición y velocidad en $t = 0$ respectivamente. En caída libre, si se desprecia el arrastre del aire, la aceleración es la de la gravedad, $a = g$.

Para digitalizar las trayectorias se usó *Tracker* [1], un programa gratuito de análisis de video del proyecto *Open Source Physics* que sirve para seguir un objeto en cada fotograma de un video y obtener su posición en función del tiempo. Está disponible en `https://physlets.org/tracker/`.

Con el eje y orientado hacia arriba, que es la convención que usa *Tracker* y la que se mantiene en todo este informe, $g \approx -9.81$ m/s^2, y tanto y como v_0 pueden ser negativos.

La ecuación (1) es el «marco teórico» cuya recuperación se busca. Hay tres observaciones sobre su estructura que resultan decisivas más adelante:

1. g aparece únicamente en el coeficiente cuadrático. Si se ajusta $y = c_0 + c_1 t + c_2 t^2$ a una trayectoria, entonces $g = 2c_2$, y ese es el estimador que se usa en todo el trabajo.



(a) Sigmoide. (b) Tangente hiperbólica. (c) ReLU.

Figura 1: Funciones de activación usadas en este trabajo. Fuente: elaboración propia.

2. Los tres coeficientes son independientes: y_0 y v_0 son propios de cada lanzamiento, mientras que g es común a todos. Una trayectoria queda determinada por la terna (y_0, v_0, g) y no por g sola.

3. Por lo tanto, t no determina y. Dos lanzamientos distintos con la misma t tienen posiciones distintas si difieren en y_0 o en v_0. Esta observación parece trivial, pero es la que dicta cómo deben construirse las entradas de la red (sección 6.3).

3.2. Red Neuronal

3.2.1. Neurona

Una red neuronal es un conjunto interconectado de nodos llamados neuronas artificiales. Estas neuronas están organizadas en capas y se comunican entre sí mediante conexiones ponderadas. Cada neurona toma la información de entrada, hace una suma ponderada, le aplica una función de activación y transmite el resultado a la siguiente neurona o, en su defecto, a la salida [2, 3, 4, 5, 6, 7].

3.2.2. Función de activación

La función de activación determina la salida de una neurona a partir de su suma ponderada. Su papel es introducir no linealidad: sin ella, una red de cualquier profundidad colapsa a una única transformación lineal [8]. Para un valor x igual a la suma ponderada de entradas y pesos, las funciones más comunes son:

1. **Función sigmoide:** lleva los valores al intervalo $(0, 1)$. Se usa sobre todo cuando se necesita predecir probabilidades, como en clasificación binaria. Se define con la ecuación (2) y su gráfica se muestra en la figura 1a,

$$f(x) = \frac{1}{1 + e^{-x}}. \tag{2}$$

2. **Tangente hiperbólica (**tanh**):** su rango es $(-1, 1)$. Es similar a la sigmoide pero centrada en el origen, lo que suele mejorar la convergencia. Está dada por la ecuación (3), y en la figura 1b se puede ver su forma,

$$f(x) = \frac{e^{x} - e^{-x}}{e^{x} + e^{-x}}. \tag{3}$$

3. **Rectificadora lineal (ReLU):** devuelve 0 para valores negativos y el valor original para valores positivos, como indica la ecuación (4); su gráfica aparece en la figura 1c,

$$f(x) = \text{máx}(0, x). \tag{4}$$

La ReLU es simple de evaluar y su derivada no se satura para $x > 0$ [9], pero tiene un modo de falla conocido. Si durante el entrenamiento el argumento de una neurona se vuelve negativo para todos los datos, su gradiente queda en cero de forma permanente y la neurona deja de aprender. Este fenómeno se conoce como *ReLU muerta* [10] y es más probable cuanto menos neuronas tenga la capa; se retomará en la sección 7.4 para explicar el comportamiento de la red con dos neuronas ocultas.

3.2.3. Capas

Hay diferentes tipos de capas neuronales:

- **Capa de entrada:** recibe los valores de entrada de la red.
- **Capas ocultas:** procesan la información de manera intermedia.
- **Capa de salida:** entrega los datos de salida de la red.

Cada neurona artificial se conecta a otras y tiene un peso y un umbral asociados. Si la salida de una neurona está por encima del umbral, se activa y envía datos a la siguiente capa; de lo contrario no propaga información [2, 3].

Para describir una red, en este informe se usa una notación con flechas en la que cada número es la cantidad de neuronas de una capa, empezando por la primera capa oculta y terminando en la de salida. Así, la red $\to 2 \to 1$ tiene una capa oculta con dos neuronas y una neurona de salida, y la red $\to 32 \to 32 \to 1$ tiene dos capas ocultas de 32 neuronas y una neurona de salida.

La cantidad de neuronas ocultas y la función de activación determinan la *capacidad expresiva* de la red, que es el tipo de funciones que la red puede representar. Con dos neuronas ocultas y activación ReLU, la red solo puede formar funciones lineales definidas a trozos, con a lo sumo dos cambios de pendiente. Con dos capas de 32 neuronas y activación tanh, en cambio, se puede aproximar cualquier función continua en un intervalo acotado con la precisión que se quiera [9]. Por eso, entre las arquitecturas de la sección 6.4 se incluyeron redes pequeñas y grandes, para poder ver cuánto afecta la capacidad de la red a los resultados.

3.2.4. Función de coste

La función de coste mide el error entre el valor estimado y el valor real durante el entrenamiento. Su minimización es el objetivo de todo el proceso [2, 3]. Sea $\hat{y}_i$ la predicción del modelo, y_i el valor real correspondiente y n el número de muestras; las funciones de coste más usadas son:

1. **Error cuadrático medio (MSE):** es la función que se minimiza durante el entrenamiento,

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^{n} (\hat{y}_i - y_i)^2. \tag{5}$$

2. **Raíz del error cuadrático medio (RMSE):** es la raíz del anterior,

$$\mathrm{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(\hat{y}_i - y_i)^2}, \tag{6}$$

con la ventaja de estar en las mismas unidades que la variable predicha, que en este trabajo son metros. Penaliza fuertemente los errores grandes.

3. **Error absoluto medio (MAE):** promedia los valores absolutos de los errores,

$$\mathrm{MAE} = \frac{1}{n}\sum_{i=1}^{n}|\hat{y}_i - y_i|. \tag{7}$$

Es más fácil de interpretar que el RMSE y menos sensible a valores atípicos.

En este punto hay que tomar en cuenta que MSE, RMSE y MAE son medidas relativas, y esto define cómo se presentan los resultados. Un RMSE de 0.5 m es excelente si los datos varían en decenas de metros y es pésimo si varían en centímetros. Por eso todo RMSE reportado en este informe se acompaña del R^2 de la ecuación (10) y de las líneas base clásicas de la sección 6.5.

Además, existen optimizadores que usan estas funciones de coste para ajustar los pesos:

- **Descenso estocástico del gradiente (SGD):** actualiza los pesos en la dirección opuesta al gradiente estimado sobre un subconjunto (lote) de los datos, $w \leftarrow w - \eta\,\partial C/\partial w$, con η la tasa de aprendizaje [9].
- **Adam** (*adaptive moment estimation*): adapta la tasa de aprendizaje de cada parámetro por separado. Para ello mantiene un promedio móvil del gradiente, m, y otro de su cuadrado, v, y actualiza cada peso según $w \leftarrow w - \eta\,\hat{m}/(\sqrt{\hat{v}} + \epsilon)$, donde $\hat{m}$ y $\hat{v}$ son esos promedios corregidos por su sesgo inicial y ϵ es una constante pequeña que evita dividir entre cero [11]. Con esto, los parámetros cuyo gradiente es grande o cambia mucho se mueven con pasos más pequeños, y los de gradiente pequeño con pasos más grandes. Es el optimizador empleado en todos los entrenamientos de este trabajo.

3.3. Herramientas Matemáticas

3.3.1. Suma ponderada

Una neurona artificial toma entradas $(x_1, x_2, \ldots, x_n)$ y pesos $(w_1, w_2, \ldots, w_n)$. La suma ponderada se calcula multiplicando cada entrada por su peso correspondiente y sumando los resultados,

$$z = w_1x_1 + w_2x_2 + \cdots + w_nx_n + b, \tag{8}$$

donde b es el sesgo, un valor constante. La suma ponderada pesa las entradas según su importancia relativa; esos pesos se adaptan durante el entrenamiento [12].

3.3.2. Mínimos cuadrados y ajuste polinómico

Dada una colección de datos, la recta de regresión es la que mejor se ajusta a ellos bajo el criterio de mínimos cuadrados, $\hat{y} = b_0 + b_1x$. El mismo criterio se generaliza a cualquier

modelo lineal en los parámetros, y en particular a la parábola que interesa aquí. Dados N pares (t_i, y_i) se construye la matriz de diseño

$$\mathbf{T} = \begin{pmatrix} 1 & t_1 & t_1^2 \\ 1 & t_2 & t_2^2 \\ \vdots & \vdots & \vdots \\ 1 & t_N & t_N^2 \end{pmatrix}, \qquad \mathbf{c} = (c_0, c_1, c_2)^{\mathsf{T}}, \tag{9}$$

y se busca el $\mathbf{c}$ que minimiza $\|\mathbf{T}\mathbf{c} - \mathbf{y}\|^2$, cuya solución es $\mathbf{c} = (\mathbf{T}^{\mathsf{T}}\mathbf{T})^{-1}\mathbf{T}^{\mathsf{T}}\mathbf{y}$. Comparando con (1) se identifica $c_0 = y_0$, $c_1 = v_0$ y $g = 2c_2$. Este ajuste es la herramienta central del trabajo, ya que se aplica tanto a los datos experimentales como a las predicciones de cada red, y la comparación entre ambos valores de g es el criterio con el que se juzga si la red capturó la física [12].

El estimador tiene una propiedad que condiciona todo el protocolo de evaluación: el ajuste solo tiene significado físico si todos los puntos (t_i, y_i) que entran en él pertenecen a una misma trayectoria. Cuando se mezclan trayectorias con distinto y_0, distinto v_0 o distinto origen temporal, el c_2 resultante ya no es $g/2$ sino un número sin interpretación. Por eso toda estimación de g que aparece en este informe, sea de los datos o de las predicciones, se hace clip por clip.

3.3.3. Coeficiente de determinación

Para juzgar si un modelo aporta información sobre la media de los datos se usa

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}, \tag{10}$$

donde $\bar{y}$ es la media muestral. $R^2 = 1$ corresponde a predicción perfecta y $R^2 = 0$ al desempeño de un modelo que devuelve siempre $\bar{y}$. Esta métrica resulta indispensable porque un RMSE «pequeño» solo significa algo al compararlo con la dispersión propia de los datos.

3.3.4. Derivadas parciales

Las derivadas parciales proporcionan información sobre cómo una función cambia respecto de cada una de sus variables independientes. Para $z = f(x_1, x_2, \ldots, x_n)$ se define

$$\frac{\partial z}{\partial a} = \lim_{\Delta a \to 0} \frac{f(x_1, \ldots, a + \Delta a, \ldots, x_n) - f(x_1, \ldots, x_n)}{\Delta a}, \tag{11}$$

donde a es cualquiera de las variables de la función [13]. En una red neuronal, las derivadas parciales de la función de coste respecto de cada peso son exactamente lo que el algoritmo de entrenamiento necesita para saber en qué dirección modificar ese peso.

3.3.5. Regla de la cadena

Si $z = f(u_1, u_2, \ldots, u_n)$ y cada u_i es función de $x_1, x_2, \ldots, x_k$, entonces [13]

$$\frac{\partial z}{\partial x_i} = \frac{\partial z}{\partial u_1}\frac{\partial u_1}{\partial x_i} + \frac{\partial z}{\partial u_2}\frac{\partial u_2}{\partial x_i} + \cdots + \frac{\partial z}{\partial u_n}\frac{\partial u_n}{\partial x_i}, \tag{12}$$

donde i toma cualquier valor entero de 1 a k. Como una red neuronal es una composición de funciones, con cada capa aplicada al resultado de la anterior, la ecuación (12) es el mecanismo que permite propagar el error desde la salida hasta la primera capa, y por eso reaparece en la sección 3.4.

3.3.6. Normalización de los datos

Antes de entrenar una red neuronal conviene llevar entradas y salidas a una escala común. La normalización z transforma cada variable según

$$\tilde{x} = \frac{x - \mu_x}{\sigma_x}, \tag{13}$$

con μ_x y σ_x la media y la desviación estándar muestrales. Con esto se evita que variables con rangos muy distintos, en este caso tiempos del orden de 10^{-1} s y posiciones del orden de 1 m, produzcan gradientes de magnitudes desbalanceadas. Los estadísticos μ_x y σ_x se calculan únicamente con los clips de entrenamiento, de manera que ninguna información del conjunto de prueba se filtre en el preprocesamiento. Las predicciones se regresan a su escala original con la transformación inversa, $x = \mu_x + \sigma_x \tilde{x}$, antes de calcular cualquier métrica física, así que todos los errores reportados en este informe están en metros.

3.4. Aprendizaje automático

3.4.1. Backpropagation

El *backpropagation*, o propagación hacia atrás de errores, es el algoritmo fundamental del aprendizaje supervisado en redes neuronales [14, 9]. Su objetivo es ajustar los pesos para minimizar el error entre las predicciones y los valores reales, y consiste en aplicar la regla de la cadena (12) desde la salida hacia la entrada: se calcula el error en la capa de salida, se propaga hacia atrás capa por capa multiplicando por las derivadas locales, y con las derivadas parciales resultantes $\partial C/\partial w$ el optimizador actualiza cada peso. El ciclo completo sobre todo el conjunto de entrenamiento es una *época*.

3.4.2. Sobreajuste y validación

Entrenar durante muchas épocas puede llevar a que la red memorice particularidades del conjunto de entrenamiento en lugar de aprender la regularidad subyacente. Para detectarlo se separan los datos en un conjunto de entrenamiento y uno de validación, y se vigila que el error de validación no empiece a crecer mientras el de entrenamiento sigue bajando [9, 15].

La forma en que se hace esa separación importa tanto como el hecho de hacerla. Cuando los datos provienen de experimentos repetidos, repartir los puntos individuales al azar hace que puntos de una misma trayectoria caigan a ambos lados de la partición; en ese caso el conjunto de validación deja de medir la capacidad de generalizar a un experimento nuevo y solo mide la interpolación dentro de experimentos ya vistos. La alternativa correcta, que es la que se empleó en este trabajo, consiste en particionar por experimento completo: cada clip entra íntegro en entrenamiento, en validación o en prueba, y nunca se reparte entre ellos.

3.4.3. Identificabilidad del modelo

Existe una condición previa a cualquier consideración de arquitectura o de entrenamiento, y es que el problema planteado tenga solución. Si se busca una función f tal que $y \approx f(x)$ y en los datos existen pares (x, y_a) y (x, y_b) con $y_a \neq y_b$, ninguna función puede reproducir ambos, de modo que el problema es *no identificable* a partir de x.

Para ese caso la teoría de la regresión da una respuesta precisa [15]. El minimizador del error cuadrático esperado es la media condicional,

$$f^\star(x) = \mathbb{E}[\, y \mid x \,], \tag{14}$$

y el error residual mínimo alcanzable es la varianza condicional $\mathbb{E}[\mathrm{Var}(y \mid x)]$, que no puede reducirse por más capacidad o más épocas que se le den al modelo. Una red suficientemente expresiva entrenada con MSE converge a (14) y no necesariamente a la ley física que generó los datos.

Esta es la consideración que gobierna el diseño experimental de la sección 6.3. Aplicada a la caída libre dice lo siguiente: si a la red se le da como única entrada el tiempo t del video y se le pide la posición y, un mismo t queda asociado a las posiciones de muchas trayectorias distintas, con alturas iniciales distintas, y la media condicional resultante no tiene por qué parecerse a ninguna parábola física. Para que la pregunta tenga respuesta, las entradas deben determinar la salida, y eso obliga a construirlas de manera que la información propia de cada trayectoria esté presente.

3.5. PyTorch

El anteproyecto contemplaba usar TensorFlow [16], la plataforma de aprendizaje automático desarrollada por Google y liberada en 2015. Durante la implementación se migró a **PyTorch** [17], una biblioteca de código abierto para aprendizaje automático en Python, mantenida por Meta AI, por dos razones prácticas. La primera es que su modo de ejecución inmediata (*eager*) permite inspeccionar tensores y gradientes con un depurador convencional, cosa que resultó decisiva para un proyecto cuyo objetivo es estudiar el interior de la red y no solamente entrenarla. La segunda es que su interfaz de programación de aplicaciones (API, en inglés) de módulos (`torch.nn`) expresa arquitecturas pequeñas y no estándar con menos código intermedio. Ambas bibliotecas implementan diferenciación automática sobre grafos de cómputo y son equivalentes para los fines de este trabajo. Los conceptos descritos en las secciones anteriores, es decir, capas lineales, activaciones, función de coste y optimizador Adam (sección 3.2.4), se traducen directamente a las clases `nn.Linear`, `nn.ReLU`/`nn.Tanh`, `nn.MSELoss` y `torch.optim.Adam`.

El entorno de trabajo se gestionó con un sistema de gestión de entornos de Python que se llama `conda` y que permite mantener entornos aislados (entorno `practicas`) y todos los scripts, datos y salidas quedaron versionados en el repositorio del proyecto, que está disponible en `https://github.com/Alejandroe0/Practicas` [18].

4. Objetivos

4.1. General

- Estudiar la teoría, ejercicios, experiencias e historia del aprendizaje automático para entender la estructura interna de las redes neuronales aplicadas a la elaboración de modelos matemáticos que describan el comportamiento de los datos de montajes experimentales de caída libre.

4.2. Específicos

- Estudiar el desarrollo histórico y aplicaciones del aprendizaje automático para la física.

- Entender el funcionamiento de las redes neuronales de aprendizaje automático.
- Construir redes neuronales básicas y entrenarlas.
- Entrenar una red neuronal con ejemplos de MRUV.

5. Justificación

Las redes neuronales y los sistemas de aprendizaje automático están cada día más presentes en la sociedad. Estas herramientas, a pesar de ser el foco de discusiones éticas y morales, son de gran ayuda en diversas áreas: artísticas, de productividad, financieras y, en especial, científicas. Las herramientas de aprendizaje automático permiten la resolución de tareas complejas en segundos. Además, se ha demostrado que son muy buenas al momento de buscar patrones y crear «modelos» matemáticos, ya que en esencia su construcción es matemática.

Tomando esto en cuenta, la idea es familiarizarse con la estructura, entrenamiento y aspectos técnicos de las redes neuronales, para posteriormente estudiar el modelo que utiliza la red para reproducir los resultados experimentales. Una vez terminado, se tendrá una herramienta que puede ayudar en la didáctica y divulgación de la ciencia y la tecnología.

Se espera que esto siente las bases para el Ejercicio de Práctica Supervisada (EPS), que pretende seguir con este estudio y generalizar algunos aspectos para diferentes marcos teóricos.

La motivación personal es adquirir experiencia en áreas de interés, como lo son la física, la tecnología y la programación. Esto será una ayuda para desenvolverme en la industria o en el mundo académico, según sea el caso, sin desaprovechar la oportunidad de contribuir con la investigación nacional y el desarrollo tecnológico y científico de Guatemala.

6. Metodología

El proyecto se desarrolló en cuatro etapas: estudio bibliográfico de redes neuronales, montaje y digitalización del experimento de caída libre, implementación y entrenamiento de las arquitecturas, y análisis de las redes entrenadas. Las tres últimas se detallan a continuación.

6.1. Montaje experimental y toma de datos

Se grabaron 609 clips de video de un cuerpo dejándose caer desde el reposo, a una frecuencia de 60 Hz ($\Delta t = 1/60$ s ≈ 16.7 ms). Cada video se segmentó en clips individuales y se digitalizó con *Tracker*, el programa de análisis de video descrito en la sección 3.1, obteniendo para cada fotograma la tupla $(t, x, y, v_x, v_y, a_x, a_y)$; las velocidades y aceleraciones son derivadas numéricas que el propio programa calcula por diferencias finitas sobre las posiciones. La escala espacial se fijó con una referencia de longitud conocida en el campo de visión, de modo que las posiciones están en metros y el eje y apunta hacia arriba.

En cuanto a los tiempos, armar el montaje experimental tomó alrededor de mes y medio. Después vinieron las grabaciones, que duraron unos tres meses y terminaron en marzo de 2025. La digitalización de los clips en *Tracker* se hizo poco a poco entre abril y agosto de 2025, y el análisis de los datos y la obtención de resultados siguieron hasta agosto de 2026, cuando se terminó el primer borrador de este informe.

De los 609 clips, 606 contienen datos utilizables, con un total de 20 210 pares (t, y). Cada clip tiene en promedio 33 puntos y una duración media de 0.54 s. Los archivos resultantes se almacenaron en formato de texto.

6.2. Preprocesamiento

Cada clip se refiere a su propio origen. Si t_0 e y_0 son el instante y la posición del primer fotograma del clip, se definen

$$\tau = t - t_0, \qquad \Delta y = y - y_0, \tag{15}$$

con lo que la ecuación (1) se convierte en

$$\Delta y = v_0\,\tau + \frac{1}{2} g\,\tau^2. \tag{16}$$

En esta forma y_0 ha desaparecido y g queda como el único parámetro compartido por todas las trayectorias. La velocidad inicial v_0, en cambio, no es la misma en todos los clips (media -0.21 m/s, desviación estándar 0.25 m/s) porque no todos empiezan en el instante exacto que se suelta el cuerpo, y por eso se conserva como variable. Se estima con un ajuste lineal a los tres primeros puntos de cada clip, por dos motivos: *Tracker* no proporciona velocidad en el primer fotograma, y usar solo el arranque evita que la estimación absorba la curvatura, que es justamente lo que se quiere medir después.

Filtro de calidad. Se descartan los clips cuyo ajuste parabólico deja un residuo superior a 8 cm. Estos corresponden a fallos de seguimiento de *Tracker*, en los que el marcador salta de objeto o se pierde, y no a trayectorias físicamente distintas. El filtro elimina 25 de los 606 clips (4 %) y deja 581 clips con 18 992 muestras.

Partición. Los clips completos se reparten en 70 %/15 %/15 % para entrenamiento, validación y prueba (407/87/87 clips), siguiendo el criterio de la sección 3.4.2. La normalización z de la ecuación (13) se ajusta únicamente con los clips de entrenamiento.

6.3. Formulaciones evaluadas

Según la sección 3.4.3, la elección de las entradas no es un detalle de implementación, sino que determina si el problema tiene una solución con interpretación física. Para medir el efecto de cada aspecto en lugar de darlo por supuesto, se evaluaron tres formulaciones:

Directa, $t \to y$. La red recibe el tiempo del video y devuelve la posición, con todos los clips concatenados. Es la formulación más ingenua posible y sirve de control: según este análisis, un mismo t está asociado a posiciones de clips distintos y el problema no queda determinado.

Referida, $\tau \to \Delta y$. Solo el cambio de origen de la ecuación (16). Mide cuánto se gana con eliminar y_0.

Completa, $(\tau, v_0) \to \Delta y$. Formulación identificable: las dos entradas determinan la salida a través de (16), con g como único parámetro por aprender.

6.4. Arquitecturas evaluadas

Se compararon tres configuraciones, elegidas para cubrir un rango amplio de capacidad expresiva con el mismo conjunto de entradas:

- **Red simple** $\rightarrow 2 \rightarrow 1$**.** Una capa oculta de dos neuronas con activación ReLU (7–9 parámetros según la formulación). Es la prueba más exigente.
- **Red media** $\rightarrow 4 \rightarrow 1$**.** Una capa oculta de cuatro neuronas con activación ReLU (13–17 parámetros).
- **Red** tanh **profunda** $\rightarrow 32 \rightarrow 32 \rightarrow 1$**.** Dos capas ocultas de 32 neuronas con activación tanh (unos 1150 parámetros). Tiene capacidad expresiva muy superior a las anteriores.

Las tres se entrenaron con optimizador Adam, tasa de aprendizaje 10^{-2}, función de coste MSE sobre los datos normalizados, hasta 800 épocas y con parada temprana sobre los clips de validación. Cada combinación de formulación y arquitectura se repitió con cinco semillas diferentes, lo que da un total de 45 entrenamientos; los resultados se reportan con la media y desviación estándar sobre esas semillas.

6.5. Líneas base

Un RMSE aislado no aporta nada al estudio, por lo que se comparó contra las tres referencias clásicas, de la más débil a la más fuerte:

1. **Predictor constante.** Devuelve siempre la media de los datos. Su RMSE es, por construcción, la desviación estándar del conjunto. Es el mínimo que cualquier modelo debe superar con holgura.
2. **MRUV con** g **global.** Se estima una única g con los clips de entrenamiento y se predice $\Delta y = v_0\tau + \frac{1}{2}g\tau^2$ en los clips de prueba. Se trata de la ecuación teórica con un solo parámetro ajustado, y es el rival de verdad: una red con cientos de parámetros que no lo iguale no está aportando nada.
3. **Ajuste parabólico por clip (oráculo).** Ajusta cada clip de prueba por separado, con tres parámetros libres por clip. No es un predictor honesto, ya que usa el propio clip que evalúa, pero marca el suelo de error alcanzable: lo que queda por debajo es ruido de medición.

6.6. Protocolo de evaluación

El desempeño se evaluó siempre sobre clips que la red no vio durante el entrenamiento, y en dos niveles.

Nivel 1: error de predicción. RMSE y MAE sobre la posición en su escala original, en metros, y el coeficiente de determinación R^2 de la ecuación (10) para situar esos errores frente a la dispersión propia de los datos y frente a las líneas base.

Equipo	Dell Precision 5770 (portátil)
Procesador	Intel Core i9-12900H, 14 núcleos / 20 hilos, hasta 4.9 GHz
Memoria	64 GiB
Sistema	Pop!_OS 24.04 LTS, núcleo 7.0.11
Lenguaje	Python 3.12
Bibliotecas	PyTorch 2.13 (compilación para CPU), NumPy 2.5, pandas 3.0
Hilos usados	4 (`torch.set_num_threads(4)`)

Cuadro 1: Equipo y entorno en que se ejecutaron todos los entrenamientos.

Nivel 2: recuperación del parámetro físico. Se ajusta la parábola de la sección 3.3.2 a las predicciones de la red para cada clip de prueba por separado y se toma $g = 2c_2$; el resultado de este nivel es la mediana de esas estimaciones. El mismo procedimiento aplicado a los datos experimentales da el valor de referencia. Como ya se advirtió en la sección 3.3.2, el ajuste se hace clip por clip. Un ajuste único sobre los puntos de todos los clips agregados mezcla trayectorias con distinto origen y distinta altura inicial, y su coeficiente cuadrático no es $g/2$ sino un número sin interpretación física.

Este segundo nivel es el que verifica el supuesto del anteproyecto: si una red «entiende» el marco teórico de la caída libre, se espera que se pueda aproximar bien la posición y, además, entregar el valor correcto de g al ajustarle una parábola a sus predicciones.

Nivel 3: estructura interna. Los dos niveles anteriores evalúan a la red por sus salidas. Como el objetivo general del anteproyecto habla de entender la estructura interna de la red, se añadieron dos análisis complementarios:

- Un **estudio del número de épocas**, que mide el tiempo acumulado de entrenamiento, el RMSE y la g recuperada en puntos de control a lo largo de un mismo entrenamiento, para valorar el coste/beneficio del entrenamiento (sección 7.5).
- Una **inspección de la red entrenada**, que escribe explícitamente la función de la red pequeña, calcula por diferenciación automática, las derivadas de la función aprendida por la red tanh y las contrasta con las presenta la ecuación (16), y examina los pesos de la primera capa (sección 7.6).

6.7. Recursos computacionales

Todos los tiempos que aparecen en este informe se midieron en el mismo equipo. Sus características se detallan en el cuadro 1 para que esas cifras sean interpretables y reproducibles.

Sobre este cuadro hay algunas aclaraciones pertinentes.

Uso de GPU. No se utilizó ningún procesamiento gráfico en los cálculos. Se instaló deliberadamente la compilación de PyTorch para CPU y el entrenamiento se limitó a cuatro hilos. Las redes de este trabajo tienen entre 9 y 1185 parámetros, y a esa escala el coste lo domina el recorrido de los datos (tiempo de lectura) y no la aritmética, así que una GPU podría ralentizar el tiempo de cómputo por la comunicación entre la gráfica y el CPU. Por lo tanto todos los tiempos del cuadro 5 deben leerse como tiempos de CPU con cuatro hilos.

	Sin filtro	Con filtro
Clips	606	581
Muestras (t, y)	20 210	18 992
Duración media [s]	0.539	0.528
Puntos por clip (media)	33.3	32.7
v_0 media [m/s]	-0.213	-0.205
v_0 desv. est. [m/s]	0.265	0.247
g por clip, mediana [m/s^2]	-9.378	$\mathbf{-9.426}$
g por clip, desv. est. [m/s^2]	2.357	1.817
Residuo del ajuste, mediana [m]	0.0350	0.0343

Cuadro 2: Propiedades del conjunto experimental antes y después de descartar los clips con residuo del ajuste superior a 8 cm. La g por clip se obtiene ajustando una parábola independiente a cada trayectoria.

Memoria. El conjunto de datos cabe holgadamente en memoria. Las 18 992 muestras con dos entradas y una salida ocupan alrededor de 228 kB en precisión simple, así que no hay carga por lotes desde disco ni ninguna limitación de memoria. Todo el conjunto reside en RAM durante el entrenamiento, y el uso máximo del proceso es de unos pocos cientos de megabytes, casi todos del propio intérprete y de las bibliotecas.

Tiempo total. El experimento principal, que incluye 45 entrenamientos (tres formulaciones, tres arquitecturas y cinco semillas, con parada temprana), tarda 138 s. El estudio de épocas, con 15 entrenamientos de hasta 800 épocas, tarda 121 s. La inspección de la red toma 5 s. En total, todo el proceso requiere poco más de cuatro minutos de cómputo en un portátil.

Una advertencia sobre reproducibilidad. Las semillas fijan la partición de los clips y la inicialización de los pesos, pero el cálculo multihilo puede hacer que el orden de las operaciones cambie ligeramente entre cada ejecución. Al repetir el experimento completo en el mismo equipo, las configuraciones estables se reproducen prácticamente igual. Entre ellas está la red tanh con la formulación completa, que es en la que se basan los resultados. En cambio, las configuraciones inestables como la red simple, afectada por el fenómeno de ReLU muerta, pueden cambiar de comportamiento entre ejecuciones. Los promedios sobre las cinco semillas de los cuadros 3 y 4 coinciden en las cuatro primeras cifras decimales, por lo que las pequeñas diferencias quedan muy por debajo de la precisión con la que se reportan los resultados.

7. Resultados

7.1. Caracterización del conjunto de datos

Antes de evaluar alguna red hay que establecer si los datos se pueden describir por algún marco teórico. El cuadro 2 resume las propiedades del conjunto antes y después del control de calidad de los datos.

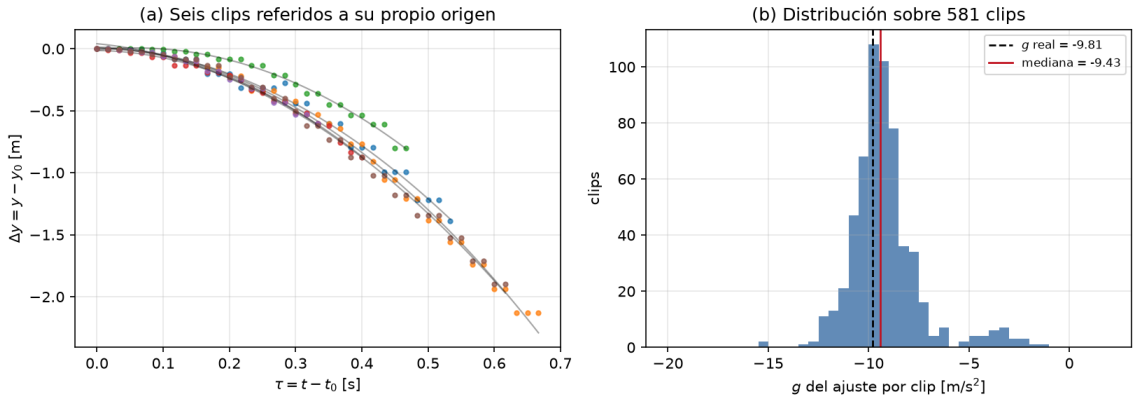


Figura 2: (a) Seis clips experimentales referidos a su propio origen, con su ajuste parabólico. Bajo esta representación todas las trayectorias arrancan en el mismo punto y comparten la misma curvatura, que es lo que fija g. (b) Distribución de la g obtenida ajustando una parábola a cada uno de los 581 clips; la mediana está a un 3.9 % del valor físico.

Los datos pasan esta primera revisión sin problema. El ajuste clásico clip por clip da

$$g_{\text{exp}} = -9.43 \text{ m/s}^2, \tag{17}$$

a un 3.9 % del valor teórico -9.81 m/s^2, lo que confirma que los datos se ajustan al marco teórico planteado. Esa diferencia del 3.9 % se interpreta como un sesgo del experimento, probablemente de la calibración de la escala espacial o de la velocidad de fotogramas antes mencionada, y afecta a las redes que se entrenaron, como a cualquier método clásico. El criterio de éxito para la red es entonces acercarse a g_{exp} y no a -9.81.

La dispersión de 1.82 m/s^2 entre clips se atribuye a la corta duración de cada uno, con una media de 0.53 s y unos 33 puntos, combinada con la resolución espacial del seguimiento. La media del residuo del ajuste, de 3.4 cm, marca el mínimo de ruido del conjunto. En la figura 2 aparecen unos cuantos clips ya referidos a su propio origen junto con la distribución completa de las 581 estimaciones de g.

7.2. Recuperación del parámetro físico

El cuadro 3 recoge el resultado principal del trabajo.

Con la formulación completa, la red tanh profunda recupera

$$g = -9.46 \pm 0.33 \text{ m/s}^2. \tag{18}$$

La comparación pertinente aquí no es respecto a -9.81 sino con la referencia experimental de -9.48 m/s^2, que es lo que el marco teórico con el método clásico extrae de estos mismos clips de prueba. La discrepancia baja a 0.2 %. La red reproduce la gravedad de este marco teórico prácticamente sin sesgo, y el 3.4 % restante frente al valor verdadero corresponde a la limitación del experimento que se discutió en la sección 7.1, común a la red y al método clásico.

La figura 3 muestra este resultado de forma gráfica, y la figura 4 muestra las distribuciones completas. Bajo la formulación directa, las estimaciones de la red y las de los datos, forman dos poblaciones separadas, mientras que bajo la formulación completa se superponen.

Formulación	Arquitectura	Parámetros	g recuperada	Error vs. -9.81
Directa	simple (2)	7	$+0.00 \pm 0.00$	100.0 %
	media (4)	13	-1.21 ± 1.67	87.6 %
	tanh 32×32	1153	-2.77 ± 0.56	71.8 %
Referida	simple (2)	7	-3.02 ± 4.14	69.2 %
	media (4)	13	-7.61 ± 0.39	22.5 %
	tanh 32×32	1153	-9.07 ± 0.28	7.6 %
Completa	simple (2)	9	-4.68 ± 4.30	52.3 %
	media (4)	17	-7.53 ± 0.57	23.3 %
	tanh 32×32	1185	$\mathbf{-9.46 \pm 0.33}$	**3.8 %**
Referencia experimental (ajuste por clip)			-9.48 ± 0.11	3.4 %

Cuadro 3: Mediana de la g recuperada ajustando una parábola a las predicciones de cada clip de prueba, en m/s^2. Media y desviación estándar sobre cinco semillas, evaluadas siempre sobre clips que la red no vio durante el entrenamiento.

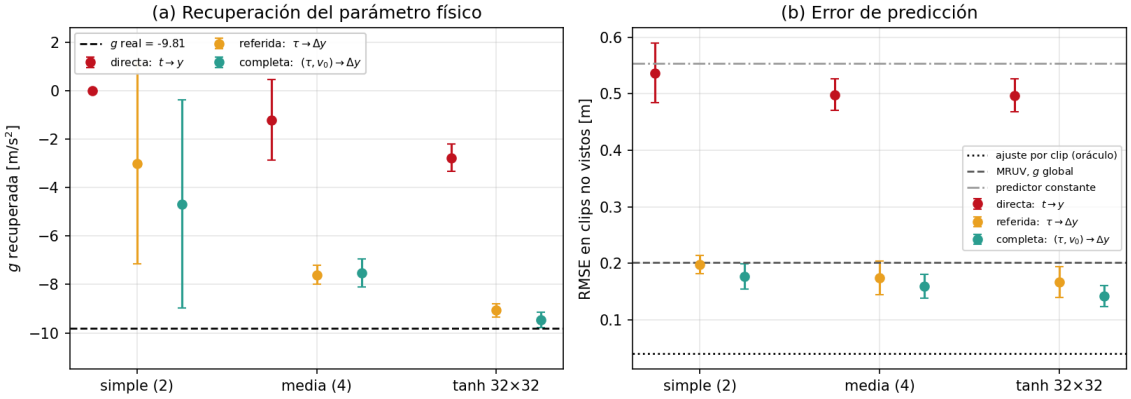


Figura 3: (a) Gravedad recuperada por formulación y arquitectura; la línea discontinua es el valor físico. (b) RMSE sobre clips no vistos, con las tres líneas base como referencia. Barras de error: desviación estándar sobre cinco semillas.

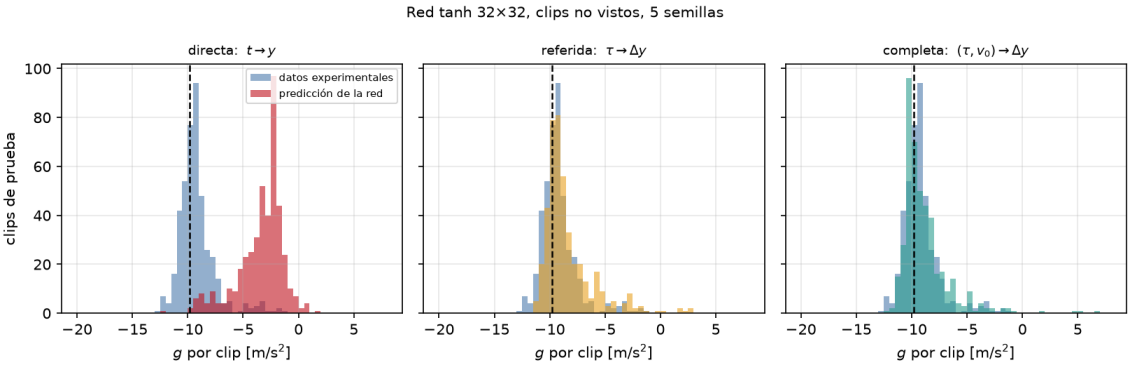


Figura 4: Distribución de la g estimada clip por clip para la red tanh 32×32; en azul la de los datos experimentales y en color la de las predicciones. Al pasar de la formulación directa a la completa las dos distribuciones dejan de estar separadas y se superponen.

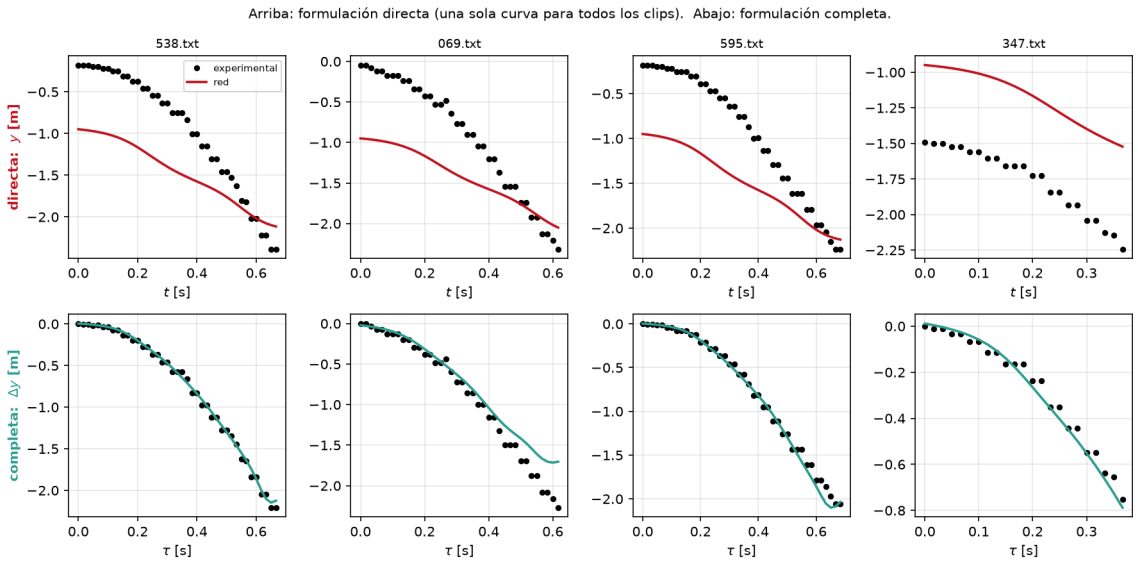


Figura 5: Cuatro clips de prueba con las predicciones de la red tanh 32×32. Arriba, formulación directa: la red no puede distinguir un clip de otro y predice esencialmente la misma curva para todos. Abajo, formulación completa: la red reproduce la trayectoria de cada clip.

En la figura 5 se puede ver qué hay detrás de estos resultados. Con la formulación directa, la red produce prácticamente la misma curva para los cuatro clips, ya que su única entrada, t, no le permite distinguir entre uno u otro clip. Es por ello que los errores pueden llegar a alcanzar un metro. En cambio, con la formulación completa, la misma arquitectura es capaz de seguir la parábola correspondiente a cada clip.

7.3. Error de predicción

El cuadro 4 sitúa el error de predicción de la red frente a las tres líneas base de la sección 6.5.

Con la formulación completa el RMSE es de 0.142 m y el R^2 de 0.93, que contrasta con el 0.553 m del predictor constante; la red explica el 93 % de la varianza. La formulación

Modelo	RMSE [m]	MAE [m]	R^2
Predictor constante $\bar{y}$	0.5534	—	0.000
Red directa, tanh 32×32	0.4970	0.4170	0.329
MRUV con g global (1 parámetro)	0.2020	0.0841	0.841
Red referida, tanh 32×32	0.1670	0.0984	0.898
Red completa, tanh 32×32	**0.1420**	**0.0728**	**0.926**
Ajuste por clip (oráculo, 3 parám./clip)	0.0401	0.0286	0.994
Suelo de ruido (residuo del ajuste)	0.0343	—	—

Cuadro 4: Error de predicción sobre clips no vistos, promediado sobre cinco semillas. Todas las cantidades en metros.

directa se queda en $R^2 = 0.33$, es decir que capta un tercio de la estructura de los datos, que es lo que se podría esperar de un modelo cuya entrada no determina la salida.

Un punto que cabe mencionar es que la red también supera al modelo físico analítico. La ecuación de MRUV con g global alcanza RMSE = 0.202 m y la red llega a 0.142 m, aproximadamente un 30 % mejor con las mismas entradas. La razón más probable es que v_0, que fue estimado a partir de tres puntos ruidosos, arrastra un sesgo sistemático que la red aprende a compensar y que la fórmula analítica no puede corregir. A pesar de ser un resultado interesante, para un sistema sin un marco teórico aceptado por la comunidad científica, puede ser difícil descifrar cuando el modelo está compensando estos sesgos sistemáticos de las observaciones.

Aún hay margen de mejora. El oráculo por clip alcanza 0.040 m, cerca del suelo de ruido de 0.034 m. La diferencia con los 0.142 m de la red se explica porque el oráculo ajusta tres parámetros por clip, mientras que la red dispone únicamente de (τ, v_0) y no puede reproducir las desviaciones individuales de cada trayectoria.

7.4. Representación de las entradas y capacidad de la red

Bajo la formulación directa, pasar de 7 a 1153 parámetros, un factor 165, apenas cambia: el RMSE baja de 0.537 a 0.497 m y la g recuperada sigue siendo inservible. Bajo la formulación completa, esa misma escalada lleva la g de -4.68 a -9.46 y el RMSE de 0.177 a 0.142 m.

Mientras el problema no sea identificable, la capacidad de la red parece no importar. Una vez que el problema sí es identificable, la capacidad pasa a ser un factor importante. Esto es lo que se expone en la sección 3.4.3. Cuando el límite está en la información disponible, ninguna arquitectura puede superarlo, porque incluso una red pequeña puede aproximarse a la solución óptima, que en este caso es la media condicional. Cuando el límite está en la representación, una red con mayor capacidad sí puede mejorar los resultados.

El comportamiento de la red $\rightarrow 2 \rightarrow 1$ con la formulación completa ($g = -4.68$, con una dispersión entre semillas de 4.30) muestra una limitación real de capacidad. Con dos unidades ReLU, la red únicamente puede representar una función lineal definida a trozos con a lo sumo dos cambios de pendiente (por limitantes de su arquitectura), por lo que no puede aproximar bien una parábola. La variación entre semillas también es coherente con el fenómeno de ReLU muerta descrito en la sección 3.2. Si una de las dos unidades queda en la región de gradiente nulo, la red funciona en la práctica como una sola neurona. Por eso, el resultado puede depender mucho de la inicialización.

La figura 6 resume las 45 corridas en un solo gráfico y muestra que ambos ejes, el error de predicción y el error en g, mejoran de la mano, aunque solo la combinación de formulación identificable y capacidad suficiente baja del 5 % de error en el parámetro físico.

7.5. Coste y beneficio del número de épocas

Entrenar durante más épocas requiere más tiempo de cómputo. Para saber si este tiempo adicional compensa, se entrenó cada arquitectura con la formulación completa y se evaluó en distintos puntos del mismo entrenamiento. Por lo tanto el tiempo reportado para la época N corresponde al tiempo real acumulado hasta llegar esa época, y la curva de error muestra cómo aprende la red, en lugar de comparar entrenamientos independientes. El cuadro 5 y la figura 7 muestran estos resultados, promediados sobre las cinco semillas.

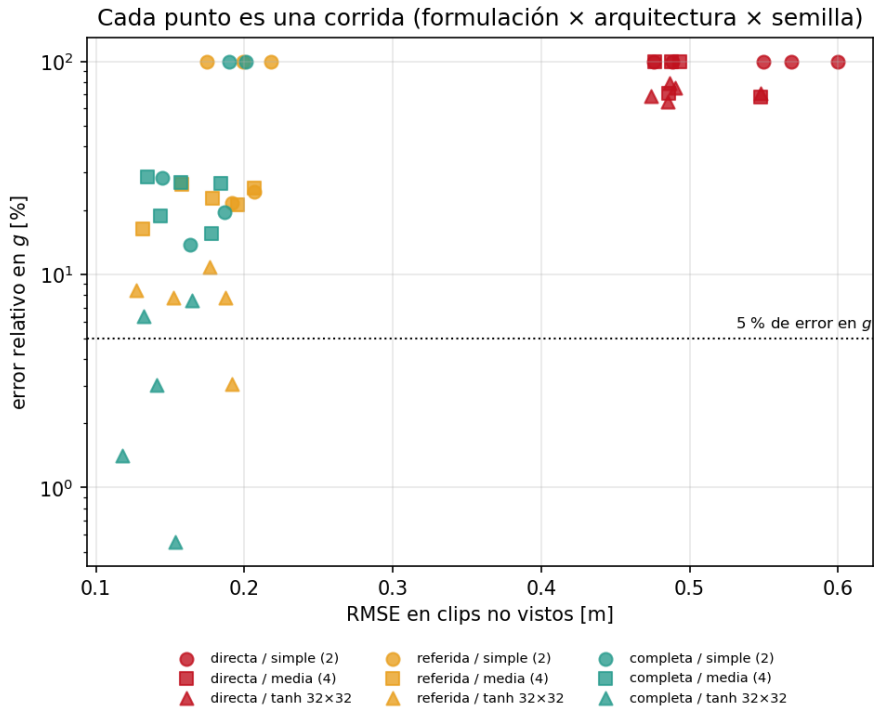


Figura 6: Error relativo en g frente al RMSE para las 45 corridas (3 formulaciones × 3 arquitecturas × 5 semillas). Las corridas con la formulación directa se agrupan arriba a la derecha; solo la formulación completa con la red tanh profunda baja del 5 % de error.

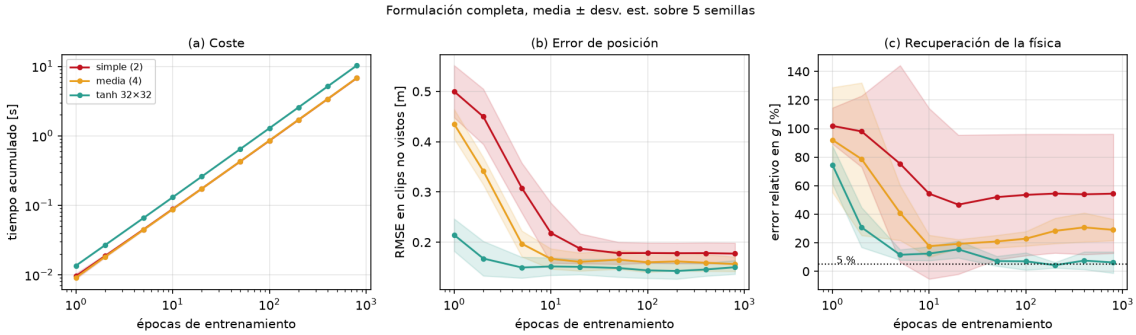


Figura 7: (a) Tiempo acumulado de entrenamiento, estrictamente lineal en el número de épocas. (b) RMSE sobre clips no vistos. (c) Error relativo en la g recuperada. Las líneas muestran la media de las cinco semillas, y la zona sombreada de cada color va desde una desviación estándar por debajo hasta una por encima de esa media; mientras más ancha es, más cambia el resultado de una semilla a otra. Nótese que el eje de épocas es logarítmico: los tramos finales de las curvas cuestan la mayor parte del tiempo total.

Arquitectura	Épocas	Tiempo [s]	RMSE [m]	R^2	Error en g
simple (2)	1	0.01	0.4999	0.095	101.9 %
	10	0.09	0.2185	0.819	54.4 %
	50	0.43	0.1783	0.884	52.0 %
	100	0.85	0.1787	0.884	53.6 %
	200	1.70	0.1782	0.884	54.5 %
	400	3.39	0.1784	0.884	54.0 %
	800	6.76	0.1775	0.885	54.4 %
media (4)	1	0.01	0.4350	0.310	92.1 %
	10	0.09	0.1670	0.898	17.7 %
	50	0.43	0.1654	0.900	20.9 %
	100	0.86	0.1602	0.906	22.9 %
	200	1.70	0.1619	0.904	28.4 %
	400	3.39	0.1593	0.908	30.8 %
	800	6.79	0.1566	0.911	29.1 %
tanh 32×32	1	0.01	0.2145	0.833	74.5 %
	10	0.13	0.1519	0.916	12.5 %
	50	0.65	0.1488	0.920	7.2 %
	100	1.29	0.1441	0.925	7.0 %
	200	**2.57**	**0.1430**	**0.926**	**4.3 %**
	400	5.13	0.1459	0.923	7.5 %
	800	10.27	0.1504	0.918	6.2 %

Cuadro 5: Tiempo acumulado de entrenamiento, error de predicción sobre clips no vistos y error relativo en g frente al número de épocas. Media sobre cinco semillas. Los tiempos corresponden al equipo del cuadro 1: CPU Intel i9-12900H con cuatro hilos, sin GPU.

El coste es lineal. El panel (a) de la figura 7 es una recta en escala doblemente logarítmica con pendiente 1; es decir, cada época cuesta lo mismo que la anterior, por lo que duplicar las épocas duplica el tiempo. Cabe notar que la red tanh, con 1185 parámetros, cuesta apenas un 50 % más por época que las de 9 y 17 parámetros; a estos tamaños el coste lo determina la lectura de los datos y no el número de parámetros.

Por otro lado, el error de posición se estanca muy pronto. Las tres redes alcanzan prácticamente su RMSE final en la época 10 o 20. De la época 50 a la 800, que representa un factor 16 en cuanto a tiempo, la red tanh mejora el RMSE en un 4 % y después empeora. Tomando en cuenta solamente el RMSE, todo lo que se entrene por encima de unas 50 épocas es tiempo perdido.

El panel (c) muestra que la física del marco teórico no aparece al mismo ritmo que baja el error, más bien puede retroceder (en este panel, igual que en los otros dos, las zonas sombreadas muestran la variación entre semillas). La red tanh necesita alrededor de 200 épocas para llegar a su mejor estimación de g, con un 4.3 % de error, es decir, cuatro veces más entrenamiento del que necesitaba para estabilizar el RMSE. La red media (4) hace lo contrario: su RMSE mejora monótonamente de 0.167 a 0.157 m entre las épocas 10 y 800, mientras su error en g empeora de 17.7 % a 29.1 %. Sigue reduciendo el error cuadrático, pero lo hace ajustando aspectos de los datos que no son la curvatura.

De aquí se desprende una consecuencia práctica: el criterio de parada no debería basarse únicamente en el error de predicción cuando el objetivo es obtener un parámetro físico, ya que ambas cantidades no siempre mejoran al mismo tiempo. En este trabajo, la parada temprana sobre clips de validación completos (sección 6.4) hace que la red tanh seleccione, en promedio, los pesos de la época 162, con un rango de 41 a 275 entre semillas. El promedio queda dentro de la ventana de 100–200 épocas, donde la estimación de g es mejor. Sin embargo, la amplitud del rango muestra que esto parece ser una coincidencia favorable de este problema y no una garantía que pueda generalizarse a otros casos.

En cuanto a la pregunta de coste y beneficio, para esta red y estos datos el punto de operación razonable está entre 100 y 200 épocas, unos 1.3 s a 2.6 s de cómputo. Las 600 épocas adicionales hasta el límite de 800 multiplican por cuatro el tiempo, empeoran ligeramente el RMSE y no mejoran la estimación de g.

7.6. Inspección de la red entrenada

El objetivo general del anteproyecto no era solo predecir bien, sino «entender la estructura interna de las redes neuronales aplicadas a la elaboración de modelos matemáticos». Ajustar una parábola a las salidas de la red, como se hizo en la sección 7.2, muestra que la red se comporta como la física. Lo que se hace en esta sección es mirar la función que la red implementa, para determinar si la física del marco teórico está también en su estructura.

Derivada	Física	Red tanh 32×32
$\partial^2(\Delta y)/\partial\tau^2$	constante $= g$	mediana -7.78, IQR $[-13.9, -2.8]$
$\partial(\Delta y)/\partial v_0$	$= \tau$	$3.55\,\tau - 0.40 \quad (R^2 = 0.26)$
$\partial(\Delta y)/\partial\tau$	$= v_0 + g\tau$	$-0.96 + 0.31\,v_0 - 5.25\,\tau \quad (R^2 = 0.35)$

Cuadro 6: Derivadas de la función aprendida por la red, en unidades físicas, frente a lo que exige la ecuación (16). Las tres tienen el signo y el orden de magnitud correctos, pero ninguna es la constante ni la recta exacta que dicta la física.

7.6.1. La red simple, escrita a mano

Con 9 parámetros la función de la red $\to 2 \to 1$ se puede escribir completa. En unidades físicas, para la semilla 0:

$$\begin{aligned} h_0 &= \text{máx}(0,\ +0.082\,\tau + 0.830\,v_0 - 0.271), \\ h_1 &= \text{máx}(0,\ -4.530\,\tau + 0.410\,v_0 + 3.056), \\ \Delta y &= 0.006\,h_0 + 0.617\,h_1 - 1.585. \end{aligned} \tag{19}$$

Se trata de una función lineal definida a trozos. En cada región del plano (τ, v_0) la pendiente es constante y solo cambia en los puntos donde una de las dos neuronas se activa o se apaga, así que hay a lo sumo dos cambios de pendiente. Por eso su segunda derivada es cero en todas partes, excepto en esos cambios de pendiente, donde no existe. No hay ningún valor de los nueve parámetros que produzca la curvatura constante que se necesita para representar g, lo cual confirma, por construcción, la limitación de capacidad de la sección 7.4. Vale la pena notar además que la primera unidad llega a la salida con un peso de 0.006, de manera que la red es en la práctica una sola neurona.

7.6.2. Las derivadas de la función aprendida

Para la red tanh 32×32, la función que aprende la red ya no se puede escribir de forma sencilla. Sin embargo, sus derivadas sí se pueden obtener mediante diferenciación automática y compararlas con lo que debería cumplirse según la ecuación (16). Si la red aprendiera realmente la relación $\Delta y = v_0\tau + \frac{1}{2}g\tau^2$, entonces sus derivadas tendrían que cumplir necesariamente

$$\frac{\partial^2(\Delta y)}{\partial\tau^2} = g\ (\text{constante}), \qquad \frac{\partial(\Delta y)}{\partial v_0} = \tau, \qquad \frac{\partial(\Delta y)}{\partial\tau} = v_0 + g\,\tau. \tag{20}$$

Estas son tres condiciones que permiten comprobar qué está haciendo la red internamente, y son más exigentes que simplemente comprobar si sus salidas se parecen a una parábola. Se evaluaron sobre los 2843 puntos experimentales de los 87 clips de prueba. No se utilizó una malla rectangular, porque esta incluiría combinaciones de (τ, v_0) que no aparecen en los datos y obligaría a la red a extrapolar. El resultado se muestra en el cuadro 6:

Los resultados, por tanto, no permiten dar una respuesta concisa. La red sí aprende parte de la estructura física, pero no la reproduce exactamente. La segunda derivada tiene el signo correcto y una mediana de -7.8m/s^2, del mismo orden que g. Sin embargo, no es constante: cambia a lo largo del dominio y presenta un rango intercuartil de $[-13.9, -2.8]$,

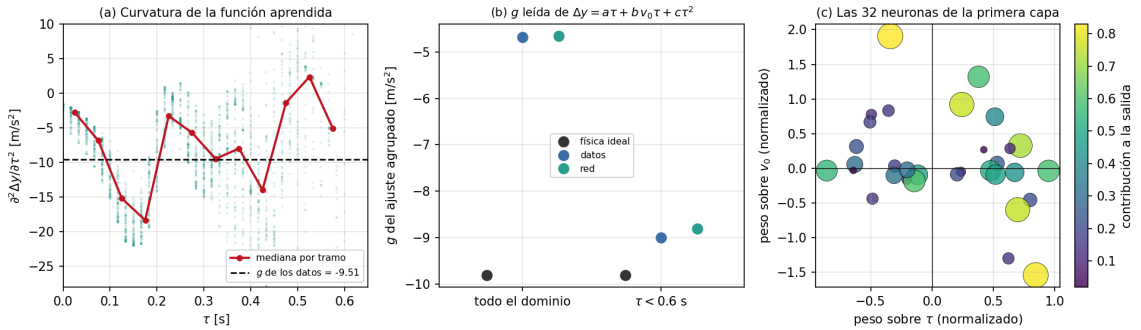


Figura 8: (a) Segunda derivada de la función aprendida, punto a punto sobre los datos de prueba, con su mediana por tramos. Oscila alrededor del valor experimental (línea discontinua) en vez de ser constante. La sombra verde azulado está formada por los valores de la segunda derivada en cada punto de prueba, dibujados semitransparentes; se ordena en columnas porque τ solo toma los instantes de muestreo del video, separados por 1/60 s. (b) g obtenida ajustando el modelo de MRUV agrupado a la salida de la red y, como control, a los datos experimentales mismos: la red sigue a los datos, no a la física ideal. (c) Pesos de entrada de las 32 neuronas de la primera capa; el tamaño y el color indican su contribución a la salida.

como se observa en el panel (a) de la figura 8. Esto indica que la red no ha aprendido una curvatura constante.

Lo que sí consigue reproducir es la curvatura promedio de cada clip. Esa es precisamente la información que recoge el ajuste parabólico por clip del nivel 2 y la que permite obtener $g = -9.46$ en el cuadro 3. Por tanto, la red consigue aprender una relación que funciona bien para estimar g, aunque su función interna no sea exactamente la ecuación física.

La figura 8 tiene otros dos paneles, que se estudian con más detalle en las dos secciones siguientes. En el panel (b) se compara la gravedad que resulta de ajustar un solo modelo de MRUV a todos los puntos juntos (ecuación (21)) en tres casos: la salida de la red, los datos experimentales y la física ideal, para dos rangos de τ. El punto de la red y el de los datos quedan casi en el mismo valor en los dos rangos, y las dos quedan igual de lejos del valor ideal. Con todo el dominio ambas están cerca de -4.7 m/s^2, y si se toma solo $\tau < 0.6$ s pasan a valer alrededor de -9 m/s^2. Esto quiere decir que la red reproduce lo que tienen los datos, con sus sesgos, y no la ley ideal (sección 7.6.3). En el panel (c) cada punto es una de las 32 neuronas de la primera capa; su posición indica los pesos que le da a τ y a v_0, y su tamaño y color, cuánto aporta a la salida. Los puntos están dispersos por todo el plano sin un patrón claro, y las neuronas que más aportan no se agrupan en ninguna zona, por lo que no hay una neurona que por sí sola represente la gravedad (sección 7.6.4).

7.6.3. La ecuación que la red aprendió, contra los datos

El resultado anterior invita a preguntarse si la desviación respecto de la física ideal es culpa de la red o si más bien es una propiedad del conjunto de datos. Para averiguarlo se ajustó el modelo

$$\Delta y = a\,\tau + b\,v_0\tau + c\,\tau^2 \tag{21}$$

cuyos coeficientes el marco teórico fija en (0, 1, $g/2$), tanto a la superficie de salida de la red como a los datos experimentales mismos. Este segundo ajuste es el que hace útil la

Dominio	Fuente	a	b	$2c = g$
Todo (2843 puntos)	Física ideal	0.000	1.000	−9.810
	Datos experimentales	−0.808	0.814	−4.674
	Red tanh	−0.979	0.486	**−4.657**
$\tau < 0.6$ s (2696 puntos)	Física ideal	0.000	1.000	−9.810
	Datos experimentales	+0.066	1.043	−8.995
	Red tanh	−0.153	0.647	**−8.807**

Cuadro 7: Coeficientes del modelo (21) ajustado a la salida de la red y a los datos experimentales, en dos dominios. El coeficiente cuadrático de la red sigue al de los datos con una diferencia inferior al 2 % en ambos casos, incluso cuando el de los datos se aleja mucho del valor físico.

comparación; sin ese control los números no significarían gran cosa.

Sobre todo el dominio, el ajuste agrupado de los datos da $g = -4.67$ m/s^2, muy lejos del valor físico, y la red da -4.66. Al restringir a $\tau < 0.6$ s, donde vive el 95 % de las muestras y donde el ajuste agrupado deja de estar contaminado por la cola de clips largos, que son pocos y los peor seguidos, los datos dan -9.00 y la red -8.81. En los dos dominios la red reproduce el coeficiente cuadrático de los datos con menos del 2 % de diferencia, sesgos incluidos.

En resumen, la red aprendió lo que está presente en los datos y no la ley física ideal. Incluso reproduce parte de los errores que aparecen en el conjunto de datos. La principal diferencia está en el coeficiente b, que mide cuánto influye la velocidad inicial. La red obtiene $b \approx 0.65$, mientras que los datos indican un valor de 1.04. Esto coincide con lo observado en la sección 7.3, donde la red supera al modelo analítico de MRUV. Una posible explicación es que la red no toma v_0 directamente, sino que reduce parcialmente su influencia, probablemente porque la estimación de v_0 a partir de solo tres puntos es ruidosa.

7.6.4. Cómo está repartida la representación

El panel (c) de la figura 8 muestra los pesos de entrada de las 32 neuronas de la primera capa. No aparece ninguna «neurona de la gravedad»: las neuronas se reparten por todo el plano (w_τ, w_{v_0}), con una razón mediana $|w_{v_0}/w_\tau| = 0.54$, y las seis de mayor contribución a la salida no comparten ninguna estructura reconocible. A esto se suma que 15 de las 32 operan casi siempre en el régimen lineal de la tanh, con activación $|\tanh| > 0.9$ en menos del 10 % del dominio.

La parábola, entonces, no está codificada en ninguna unidad ni en ningún peso identificable, sino en la combinación de muchas neuronas casi lineales, ninguna de las cuales es interpretable por separado. Dicho de otra manera, la red recupera correctamente el parámetro físico, pero al hacerlo no construye internamente ninguna representación de la ecuación de movimiento que un físico pueda leer. La física está en el comportamiento agregado de la red y no en su estructura.

7.7. Síntesis

Reuniendo la evidencia:

- Los datos experimentales contienen la física buscada: ajustados clip por clip dan $g_{\text{exp}} = -9.43$ m/s^2 (cuadro 2).
- La red tanh profunda con entradas (τ, v_0) recupera $g = -9.46 \pm 0.33$ m/s^2 sobre clips no vistos, a un 0.2 % de lo que el método clásico obtiene de esos mismos datos (cuadro 3).
- Su RMSE de 0.142 m ($R^2 = 0.93$) supera en un 30 % al modelo analítico de MRUV con una única g ajustada (cuadro 4).
- Las predicciones siguen la trayectoria propia de cada clip (figura 5), y la distribución de las g predichas se superpone con la de los datos (figura 4).
- El coste de entrenar es lineal en el número de épocas, y el beneficio se agota pronto: el RMSE se estanca hacia la época 20 y la mejor estimación de g se alcanza hacia la 200 (cuadro 5).
- Al mirar dentro de la red, el ajuste agrupado de su salida reproduce el coeficiente cuadrático de los datos con menos del 2 % de diferencia, pero su curvatura punto a punto no es constante y ninguna neurona es interpretable por separado (cuadros 6 y 7).

El supuesto del anteproyecto queda verificado: una red neuronal entrenada con datos experimentales de caída libre sí permite recuperar el modelo físico subyacente. Lo que el estudio muestra además es que ese resultado depende de manera crítica de cómo se estructuren las entradas, ya que la misma arquitectura, con los mismos datos y el mismo entrenamiento, entrega o no entrega la gravedad según que la representación haga identificable el problema.

8. Limitaciones del estudio

Las siguientes son propiedades del conjunto de datos y del procedimiento, y acotan lo que puede lograrse con ellos, con red o sin ella.

1. **Sesgo experimental en g.** El método clásico aplicado a estos datos da -9.43 m/s^2, un 3.9 % por debajo del valor físico. Ese sesgo, que probablemente proviene de la calibración de la escala espacial o de la velocidad de fotogramas, afecta por igual a la red y al ajuste clásico, y fija el techo de exactitud alcanzable con este conjunto.
2. **Calidad de la digitalización.** La dispersión de 1.82 m/s^2 en la g experimental por clip indica cuantización a nivel de píxel en el seguimiento. El filtro de calidad descarta los peores casos, pero no elimina el ruido de fondo, que se refleja en el residuo mediano de 3.4 cm.
3. **Duración de los clips.** Con 0.53 s de media, el término cuadrático de la ecuación (16) contribuye poco frente al ruido de medición, lo que hace intrínsecamente difícil estimar g a partir de un clip aislado.
4. **Estimación de v_0.** Se obtiene de los tres primeros puntos de cada clip y arrastra el ruido de esas mediciones. Que la red supere al modelo analítico de MRUV (sección 7.3) sugiere que compensa un sesgo de esa estimación, pero eso no se comprobó de forma directa.

5. **La física no es legible en la estructura de la red.** Como muestra la sección 7.6, la red recupera g correctamente pero su curvatura punto a punto no es constante y la representación está repartida entre muchas neuronas casi lineales, ninguna interpretable por separado. La inspección permite afirmar qué no hay dentro de la red, pero no extraer de ella una forma cerrada del modelo físico.

6. **El criterio de parada se apoya en el error de predicción.** La sección 7.5 muestra que el RMSE y el error en g no cambian de la misma forma durante el entrenamiento (uno puede seguir bajando mientras el otro sube), de modo que la parada temprana sobre el error de validación no garantiza detenerse en el óptimo del parámetro físico. En este problema ambos coincidieron aproximadamente, pero eso no es general.

9. Conclusiones

1. Se construyó un conjunto de datos experimental propio de caída libre: 609 clips grabados a 60 Hz, 606 utilizables y 581 tras el filtro de calidad, con 18 992 pares (t, y) digitalizados con *Tracker*. El conjunto contiene la física buscada, ya que un ajuste cuadrático clip por clip recupera $g = -9.43$ m/s^2, a un 3.9 % del valor esperado.

2. Se implementaron y entrenaron en PyTorch tres arquitecturas de red neuronal de capacidad creciente, bajo tres formulaciones de las entradas y con cinco semillas cada una, con partición por clip completo, parada temprana y tres líneas base clásicas. Con ello se cumplieron los objetivos específicos relativos a la construcción, entrenamiento y estudio de redes neuronales.

3. El supuesto del anteproyecto se verifica. Con entradas (τ, v_0) y salida Δy, la red tanh profunda recupera $g = -9.46 \pm 0.33$ m/s^2 sobre clips que no vio durante el entrenamiento. Frente al -9.48 m/s^2 que el método clásico extrae de esos mismos datos, la discrepancia es del 0.2 %. La diferencia restante con -9.81 corresponde al sesgo experimental y no a un fallo del modelo.

4. En términos de predicción, la red alcanza RMSE = 0.142 m y $R^2 = 0.93$ sobre clips no vistos, frente a los 0.553 m del predictor constante.

5. La red supera al modelo analítico: RMSE de 0.142 m frente a 0.202 m de la ecuación de MRUV con una única g ajustada, y con las mismas entradas. Este es el punto del proyecto en que la red aporta algo que el método clásico no proporciona, verosímilmente por compensar el sesgo de la estimación de v_0.

6. La representación de las entradas y la capacidad de la red son dos límites distintos y sucesivos. Con la formulación directa, multiplicar por 165 el número de parámetros no mejora la estimación de g; con la formulación completa, esa misma escalada la lleva de -4.68 a -9.46 m/s^2. Cuando el límite es de información, ninguna arquitectura lo supera; cuando es de representación, la capacidad pasa a ser decisiva.

7. El beneficio de entrenar más se agota mucho antes que el coste. El tiempo crece linealmente con las épocas, mientras que el RMSE alcanza su valor final hacia la época 20 y la mejor estimación de g hacia la 200. Para esta red y estos datos el punto de operación razonable son 100–200 épocas, unos 2 s de cómputo; las 600 épocas siguientes cuadruplican el tiempo y no mejoran nada.

8. El error de predicción y la recuperación de la física no van al mismo ritmo, e incluso pueden ir en sentidos opuestos. La red media (4) mejora su RMSE de 0.167 a 0.157 m entre las épocas 10 y 800 mientras su error en g empeora del 17.7 % al 29.1 %. Un criterio de parada basado únicamente en el error de validación no garantiza, por tanto, detenerse donde el parámetro físico es mejor.

9. La red aprende lo que los datos contienen, con sesgos y todo. Al ajustar el modelo $\Delta y = a\tau + b\, v_0\tau + c\tau^2$ a la salida de la red y, como control, a los datos experimentales, el coeficiente cuadrático de la red sigue al de los datos con menos del 2 % de diferencia en los dos dominios examinados, incluso donde el de los datos se aparta mucho del valor físico.

10. La física está en el comportamiento de la red y no en su estructura. La curvatura de la función aprendida tiene el signo y el orden de magnitud correctos pero no es constante, y la representación está repartida entre muchas neuronas casi lineales sin que ninguna sea interpretable por separado. La red recupera g correctamente sin construir internamente nada que se parezca a la ecuación de movimiento. Extraer una forma cerrada exigiría arquitecturas diseñadas para ello, y esa es la continuación natural del trabajo.

11. Una red neuronal entrenada con MSE converge a la media condicional de los datos. Que esa media condicional coincida o no con la ley física depende enteramente de cómo se hayan estructurado las entradas, de manera que la condición de identificabilidad es previa a cualquier consideración de arquitectura. Este es el aprendizaje central del proyecto y la base sobre la que se plantea su continuación.

12. Desde el punto de vista metodológico, validar un modelo de aprendizaje automático contra una teoría física exige que el valor de referencia se verifique con el mismo rigor que el valor predicho. Aquí eso significó estimar g clip por clip en ambos lados de la comparación y contrastar el resultado contra líneas base clásicas explícitas.

10. Trabajo futuro

Las siguientes líneas quedan abiertas, ordenadas de la más inmediata a la más ambiciosa, y constituyen la continuación natural hacia el EPS.

1. Cerrar la brecha con el oráculo. La red alcanza RMSE = 0.142 m frente a los 0.040 m del ajuste por clip. Parte de esa diferencia es irreducible, porque el oráculo dispone de tres parámetros por clip, pero conviene medir cuánta. Hay dos vías: predecir directamente los coeficientes (c_0, c_1, c_2) de cada clip en lugar de puntos sueltos, y estimar v_0 con más robustez, con un ajuste local en vez de tres puntos.

2. Diagnosticar el sesgo experimental. El método clásico da -9.43 m/s^2 sobre estos datos, un 3.9 % por debajo del valor real. Antes de perseguir mejoras en el modelo conviene identificar si el sesgo viene de la calibración de la escala espacial, de la velocidad de fotogramas declarada o del arrastre del aire, y corregirlo en la toma de datos.

3. Mejorar la calidad de los datos. Grabar clips más largos o desde mayor altura, para que el término cuadrático domine sobre el ruido, y aumentar la resolución espacial o

la frecuencia de muestreo. Con 0.53 s de media por clip, la señal cuadrática es débil frente a la cuantización de píxel.

4. Hacer la red interpretable a propósito. La inspección de la sección 7.6 muestra que la física no está codificada de forma legible en los pesos. Si se quiere que lo esté hay que forzarlo, por ejemplo con redes que tengan una capa de cuello de botella de una sola unidad, con activaciones polinómicas, o aplicando regresión simbólica sobre la función aprendida. Cualquiera de las tres convertiría «la red predice bien» en «la red entrega una ecuación», que es el objetivo natural del EPS.

5. Un criterio de parada basado en la física. Dado que el error de predicción y el error en g no cambian de la misma forma durante el entrenamiento (sección 7.5), tiene sentido explorar una parada temprana que vigile directamente la g estimada sobre los clips de validación en lugar del MSE.

6. Explorar el aprendizaje con restricciones físicas. Incorporar el residuo de la ecuación de movimiento a la función de coste (*physics-informed neural networks*) permitiría comparar cuánta física aporta la arquitectura frente a cuánta aporta la función de coste, y previsiblemente reduciría la dispersión entre semillas.

7. Generalizar a otros marcos teóricos. Con el protocolo ya validado sobre la caída libre, extender el estudio a movimiento en dos dimensiones, a sistemas con fricción y a osciladores, que es el objetivo planteado para el EPS. El procedimiento desarrollado aquí, esto es, formular el problema de modo que sea identificable, comparar contra una línea base clásica y estimar el parámetro físico caso por caso, es directamente transferible a esos sistemas.

11. Bibliografía

Referencias comentadas

Todo lo pertinente a teoría física fue tomado de Blatt [19]. La referencia estadística predeterminada, para regresión, mínimos cuadrados y métricas de error, es el libro de Triola [12]. La parte matemática, es decir, derivadas parciales y regla de la cadena, se consultó en el libro de cálculo de Zill [13]. El canal de YouTube DotCSV es una referencia audiovisual bastante completa en cuanto a redes neuronales se refiere [4, 5, 6, 7], y complementa la introducción de Antonio [2] y la documentación divulgativa de IBM [3]. Para consultas técnicas rápidas sobre funciones de activación se usó la página de Wikipedia correspondiente [8]. Las ayudas para entender el manejo de bibliotecas de aprendizaje profundo se tomaron de Kapoor, Gulli y Pal [20] y, como documentación complementaria, del libro de Martínez [21]. La documentación de TensorFlow [16] se consultó durante la fase inicial, antes de la migración a PyTorch descrita en la sección 3.5.

Referencias

[1] D. Brown, R. Hanson, and W. Christian, "Tracker: Video analysis and modeling tool." Open Source Physics, `https://physlets.org/tracker/`, 2024.

[2] J. Antonio, "Introducción a las redes neuronales: explicación completa en español," 08 2023.

[3] IBM, "¿qué son las redes neuronales? — ibm."

[4] C. Santana, "¿qué es una red neuronal? parte 1 : La neurona — dotcsv," 03 2018.

[5] C. Santana, "¿qué es una red neuronal? parte 2 : La red — dotcsv," 05 2018.

[6] C. Santana, "¿qué es una red neuronal? parte 3 : Backpropagation — dotcsv," 10 2018.

[7] C. Santana, "¿qué es una red neuronal? parte 3.5 : Las matemáticas de backpropagation — dotcsv," 10 2018.

[8] W. Contributors, "Activation function," 11 2018.

[9] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016. `https://www.deeplearningbook.org`.

[10] L. Lu, Y. Shin, Y. Su, and G. E. Karniadakis, "Dying ReLU and initialization: Theory and numerical examples," *Communications in Computational Physics*, vol. 28, no. 5, pp. 1671–1706, 2020.

[11] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *3rd International Conference on Learning Representations (ICLR)*, 2015. arXiv:1412.6980.

[12] M. F. Triola, *Estadística*. Pearson Educación de México, S.A. de C.V., 2018.

[13] D. G. Zill, *Cálculo con geometría analítica*. Grupo Editorial Iberoamérica, 1985.

[14] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, "Learning representations by back-propagating errors," *Nature*, vol. 323, pp. 533–536, 1986.

[15] T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning*. Springer, segunda ed., 2009.

[16] Google Brain Team, "Por qué tensorflow," 2014.

[17] A. Paszke, S. Gross, F. Massa, A. Lerer, *et al.*, "PyTorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems 32*, pp. 8024–8035, 2019.

[18] C. A. Barillas Segura, "Repositorio del proyecto de prácticas: redes neuronales para la inferencia de modelos físicos." `https://github.com/Alejandroe0/Practicas`, 2026.

[19] F. J. Blatt, *Fundamentos de Física*. Prentice-Hall Hispanoamericana, S.A., 1991.

[20] A. Kapoor, A. Gulli, and S. Pal, *Deep Learning with TensorFlow and Keras*. Packt Publishing Ltd., 2022.

[21] J. Martínez, *TensorFlow 2.0 Computer Vision Cookbook*. Packt Publishing Ltd., 2021.